I checked the paper you mentioned and compared its architecture with related SRAM-based compute-in-memory (CIM) research. The most important finding is that your proposed paper needs to reduce a specific remaining latency bottleneck, rather than simply move more adders inside SRAM.

1. Verification of the reference paper

SRAM Based Digital Custom Compute Engine for Improved Area Efficiency of AI Hardware

Authors: Narendra Singh Dhakad and Santosh Kumar Vishvakarma

2026 preprint

arXiv:2605.16161

The paper was submitted to arXiv on May 15, 2026. It describes a digital SRAM-CIM architecture using a 10-transistor (10T) SRAM cell for XNOR-based computation and a compact 14T full adder for accumulation. The authors report improved routing and area efficiency. The arXiv record is the appropriate source for the title, authors, and publication status.

arxiv.org
+1

Read the original paper on arXiv

Important publication distinction: the record I verified is an arXiv preprint. I have not established that this specific paper has been accepted by a peer-reviewed SCI-indexed journal. It should not be cited as a journal article unless a separate journal publication can be verified.

What the reference paper contributes

Feature	Reference architecture
Memory	10T SRAM
Multiplication	XNOR-based binary neural network computation
Accumulation	Full adder shared between adjacent rows, followed by an adder tree
Main optimization	Reduced routing and adder-tree area
Reported result	2.67× area-efficiency improvement over the stated comparison baseline

The paper reports reducing routing tracks from 128 to 72 for its stated 16 × 8 example. These are the reference authors' reported results, not independently reproduced measurements.

ResearchGate
+1

What your new paper should improve

A useful direction is to investigate hierarchical carry-save accumulation for reducing serial carry propagation. However, the idea is not automatically novel: other digital SRAM-CIM research has already investigated custom accumulation circuits and approximate adder trees. For example, LPAE-CIM appeared in Microelectronics Journal in December 2025, while DREAM-CIM was published in IEEE Transactions on Circuits and Systems for Artificial Intelligence in 2025.

ScienceDirect
+1

Therefore, the manuscript below is framed as a proposed architecture and experimental study, with no invented simulation results. Its novelty must be confirmed through a broader literature review and circuit-level validation before submission.

2. SCI-style manuscript draft

Proposed research article • Digital SRAM-CIM • Latency optimization • 65 nm evaluation target

One Method to Reduce the Latency in SRAM-Based Compute-in-Memory

Abstract

Static random-access memory-based computing-in-memory (SRAM-CIM) is a promising approach for reducing the data-movement overhead associated with artificial intelligence (AI) inference. However, the latency of digital SRAM-CIM architectures remains affected by partial-product accumulation, carry propagation, interconnect delay, and the transfer of intermediate results between the memory array and peripheral computing circuits. Recent digital SRAM-CIM architectures have reduced routing overhead by integrating arithmetic operations into the memory array, but the remaining accumulation path may limit the latency improvement achievable through further integration alone.

This paper proposes a latency-oriented accumulation architecture for digital SRAM-CIM based on hierarchical carry-save reduction. The proposed architecture aims to reduce the number of serial carry-propagation stages required to accumulate multiple binary multiply results. Rather than using a conventional ripple-carry structure at every intermediate accumulation stage, the

design groups partial products into local reduction units and employs carry-save arithmetic to compress multiple operands before a final carry-propagate addition. The proposed architecture is intended to be evaluated against a 10-transistor SRAM-based XNOR computation architecture with integrated first-stage accumulation.

A 65 nm CMOS implementation is proposed for a controlled comparison with the reference architecture. Circuit-level evaluation will measure critical-path latency, energy per operation, silicon area, routing requirements, and area-delay product. Functional verification will use binary neural network (BNN) workloads, with an additional assessment of inference accuracy where appropriate. Since the proposed architecture has not yet been simulated or fabricated, numerical improvements are not claimed. The study establishes a reproducible evaluation framework for determining whether hierarchical carry-save accumulation can reduce latency without imposing unacceptable area and energy overhead.

Keywords: SRAM-based computing-in-memory; digital CIM; latency optimization; carry-save adder; binary neural network; XNOR computation; partial-sum accumulation; CMOS circuit design.

1. Introduction

Artificial intelligence workloads increasingly require efficient execution of multiply-accumulate (MAC) operations. In conventional computing architectures, model parameters are stored in memory and transferred to arithmetic units for processing. Repeated data movement consumes energy and can limit throughput, particularly in edge devices with constrained power and memory bandwidth.

Computing-in-memory addresses this problem by performing selected computations within or near the memory array. SRAM-based CIM is particularly attractive because SRAM is compatible with conventional CMOS processes and can support fast access to stored data. Digital SRAM-CIM is also useful where deterministic logic operation and compatibility with digital design flows are priorities.

Despite these advantages, digital SRAM-CIM does not eliminate all sources of computational latency. The memory array must still generate partial results, and the resulting values must be combined to produce final outputs. In many architectures, the accumulation network introduces additional logic depth, interconnect capacitance, and routing complexity. Carry propagation can become a critical contributor when partial sums are combined through cascaded carry-propagate adders.

Dhakad and Vishvakarma proposed a digital custom compute engine that combines 10T SRAM-based XNOR computation with a compact full-adder structure to improve area efficiency and reduce routing overhead. Their architecture integrates a first accumulation stage into the memory array and uses an adder tree to obtain the final result. This work provides a relevant baseline for investigating the remaining accumulation delay.

The present study focuses on the accumulation network rather than replacing the underlying SRAM cell. It proposes a hierarchical carry-save reduction scheme that postpones full carry propagation until the final addition. The central hypothesis is that reducing the number of serial carry-propagation stages can lower critical-path latency for suitable accumulation widths and array configurations.

1.1 Research objectives

The objectives are:

1. To identify the accumulation-related critical path in a digital SRAM-CIM architecture.
2. To develop a hierarchical carry-save reduction network compatible with SRAM-based XNOR computation.
3. To compare the proposed architecture with a conventional accumulation tree under equivalent process, voltage, array-size, and workload conditions.
4. To quantify latency, energy, area, routing, and numerical correctness.
5. To determine the workload and array configurations for which the proposed method offers a measurable advantage.

1.2 Contributions

The proposed contributions are:

- A hierarchical accumulation architecture that uses carry-save compression to reduce serial carry propagation.
- A design methodology for integrating local reduction units with a digital SRAM-CIM macro.
- A controlled evaluation framework separating the effects of arithmetic topology, routing, and memory-cell design.
- An assessment of latency and energy trade-offs under binary neural network workloads.

The proposed contribution should be considered a hypothesis until the architecture is implemented and compared with relevant prior work. In particular, the study must establish how its reduction topology differs from previously published custom accumulation circuits.

2. Background and related work

2.1 SRAM-based digital compute-in-memory

A conventional SRAM-CIM architecture stores neural-network weights in SRAM cells and performs selected logical operations during memory access. For binary neural networks, XNOR-based computation is commonly used to compare binary weights and activations.

For binary values represented as (W_i,X_i\in{-1,+1}), their product can be represented by an XNOR operation after an appropriate binary encoding. The resulting bit-level products must then be accumulated or converted into a count of matches.

For a vector of length (N), the dot product is

[
Y=\sum_{i=1}^{N}W_iX_i.
]

If (M) is the number of matching binary elements, then

[
Y=2M-N.
]

Thus, the accumulation network can be implemented using binary counting, signed arithmetic, or a related reduction structure, depending on the selected encoding.

2.2 Accumulation latency

A ripple-carry adder propagates carry information from one bit position to the next. Its worst-case delay increases approximately linearly with the operand width under a simplified gate-delay model.

An adder tree can reduce the number of operand-combination levels compared with serial accumulation. However, each tree node may still require carry propagation. Its actual delay depends on the adder topology, fan-out, wiring, loading, and physical placement.

Carry-save addition offers an alternative for combining three operands. It produces sum and carry vectors without propagating carry across the complete word. Repeated carry-save compression can reduce several operands to two vectors, which are combined by a final carry-propagate adder.

2.3 Relationship to previous work

The reference work, *SRAM Based Digital Custom Compute Engine for Improved Area Efficiency of AI Hardware*, integrates a compact full-adder structure into the SRAM-CIM array and reduces the amount of data routed to the external adder tree. The proposed study retains the concept of in-memory XNOR computation as a baseline but investigates a different aspect: the organization of the subsequent accumulation network.

Other published approaches have also investigated approximate adder trees, custom accumulation circuits, and pipelined SRAM-CIM architectures. Accordingly, the novelty of the proposed method cannot be established solely by using carry-save adders. It must be

demonstrated through a specific topology, placement strategy, or latency–area–energy advantage that is not already present in closely related designs.

3. Proposed architecture

3.1 System overview

The proposed digital SRAM-CIM system consists of five functional stages:

1. SRAM weight storage and access.
2. XNOR-based binary multiplication.
3. Local carry-save reduction.
4. Hierarchical carry-save compression.
5. Final carry-propagate addition and output generation.

The architecture retains the memory-array computation concept and changes the organization of the accumulation network.

3.2 Mathematical formulation

Let the binary multiplication stage generate partial products (p_i). Depending on the encoding, these may be represented as match indicators or signed values.

For a general arithmetic formulation, let the partial products be (P_1,P_2,\ldots,P_N). The required output is

[
Y=\sum_{i=1}^{N}P_i.
]

A conventional implementation repeatedly combines operands using carry-propagate adders. The proposed implementation first compresses multiple operands using carry-save units.

For three (B)-bit operands (A,B,C), a carry-save stage generates

[
S=A\oplus B\oplus C
]

and

[
C_{\mathrm{shift}}=
\left[(A\land B)\lor(A\land C)\lor(B\land C)\right]\ll1.
]

Here, the operations are bitwise operations, and the shifted carry vector represents carry contributions to the next bit positions. Subject to the chosen width and overflow convention,

[
A+B+C=S+C_{\mathrm{shift}}.
]

The crucial property is that the sum and carry vectors are generated locally without a word-wide carry-propagation chain.

3.3 Hierarchical reduction

The proposed method divides the partial products into groups. Each group reduces its operands through carry-save stages. The resulting sum and carry vectors are forwarded to higher-level reduction stages.

For example, the first level processes local groups of partial products. The next level combines the compressed outputs of these groups. Further levels continue until only two operands remain.

The final two operands are added using a carry-propagate adder. The design therefore postpones global carry propagation until the end of the reduction process.

The exact number of levels depends on the number of partial products and the compression schedule. A balanced reduction network can reduce the number of sequential compression stages relative to a poorly balanced serial structure, but its advantage over an optimized adder tree must be measured rather than assumed.

3.4 Integration with SRAM-CIM

The design should compare at least two integration strategies:

Architecture A — baseline: 10T SRAM-based XNOR computation, the reference-style first accumulation stage, and a conventional downstream adder tree.

Architecture B — proposed: the same SRAM-based XNOR computation and equivalent initial accumulation, followed by hierarchical carry-save compression and a final carry-propagate adder.

This comparison isolates the impact of the accumulation topology. If the SRAM cell, input precision, operating voltage, or process node differs between the designs, the comparison will not isolate the intended contribution.

The physical implementation should minimize communication between distant reduction units. Local compression can reduce the number of full-width intermediate operands, but extra local logic and wiring may increase area and capacitance. Placement and routing are therefore part of the evaluation, not merely implementation details.

4. Circuit design considerations

4.1 Arithmetic correctness

All arithmetic stages must preserve the required numerical result. For binary neural networks, the implementation must account for the relationship between XNOR match counts and signed dot products.

The designer must define operand width, sign representation, intermediate width, and overflow behavior. Truncation should not be introduced unless it is part of the specified numerical algorithm.

4.2 Final carry-propagate adder

Carry-save compression removes repeated carry propagation between intermediate operands but does not eliminate the need for a final addition. The final adder may use ripple-carry, carry-lookahead, prefix, or another appropriate topology.

The choice should be made using the target width, available cell library, area constraint, and measured post-layout delay. A faster final adder may increase area and wiring, so its benefit must be evaluated at the system level.

4.3 Pipeline and clocking considerations

The proposed reduction stages can potentially be pipelined. However, adding registers may increase area, clock power, and the number of cycles required to produce the first result.

The evaluation must distinguish:

- **Latency:** elapsed time from accepting an input to producing its corresponding output.
- **Throughput:** number of completed operations per unit time after the pipeline is filled.
- **Cycle count:** number of clock cycles between input acceptance and output availability.

A design that increases throughput by pipelining does not necessarily reduce single-operation latency. Both quantities should be reported separately.

5. Experimental methodology

5.1 Technology and tools

A 65 nm CMOS process is proposed to support comparison with the reference architecture. The final paper should identify the actual process design kit, standard-cell library, transistor models, supply voltage, temperature, extraction settings, and design tools used.

The following workflow is recommended:

1. Develop the RTL or gate-level functional description.
2. Verify arithmetic correctness using exhaustive tests for small configurations and randomized tests for larger configurations.
3. Synthesize both architectures using the same target library and constraints.
4. Perform static timing analysis.
5. Complete placement and routing.
6. Extract parasitic resistance and capacitance.
7. Perform post-layout timing and power analysis.
8. Repeat measurements across relevant operating conditions.

A 65 nm target is a proposed evaluation choice, not a claim that the architecture has already been implemented at that node.

5.2 Benchmark configurations

The study should include multiple reduction widths and array sizes rather than one isolated configuration. The baseline and proposed architectures must use equivalent input data, numerical precision, output precision, and memory-array behavior.

For each configuration, measure:

- Critical-path delay.
- Input-to-output latency.
- Maximum operating frequency, where meaningful.
- Energy per completed operation.
- Dynamic and leakage power.
- Cell area and total macro area.
- Routing area or an explicitly defined routing-complexity metric.
- Output correctness and, where applicable, neural-network inference accuracy.

5.3 Performance metrics

Latency reduction is defined as

[
R_L=
\frac{L_{\mathrm{base}}-L_{\mathrm{prop}}}
{L_{\mathrm{base}}}\times100%,
]

where (L_{\mathrm{base}}) and (L_{\mathrm{prop}}) are the measured latencies of the baseline and proposed architectures.

The energy ratio is

$$
R_E=\frac{E_{\mathrm{prop}}}{E_{\mathrm{base}}},
$$

and the area ratio is

$$
R_A=\frac{A_{\mathrm{prop}}}{A_{\mathrm{base}}}.
$$

For a combined latency–area metric, define

$$
P_{AD}=A\times L.
$$

A lower P_{AD} indicates a better area–latency product under the same area and latency definitions. This metric should supplement, not replace, the individual measurements.

Energy-delay product can similarly be evaluated as

$$
EDP=E\times L.
$$

The paper should specify whether energy is measured per operation, per inference, or per clock cycle.

6. Results and discussion

This section must be completed using actual synthesis, timing, and post-layout measurements. No numerical result should be inserted before the corresponding experiment has been completed.

6.1 Functional verification

Report whether both architectures produce identical outputs for all test cases within the defined arithmetic precision. Any differences must be explained in terms of encoding, overflow, truncation, or implementation errors.

6.2 Latency comparison

Compare the critical paths of the baseline and proposed architectures. The analysis should identify the dominant delay components, including XNOR generation, local accumulation, carry-save compression, final carry propagation, and interconnect.

If the proposed design reduces logic depth but increases routing delay, the post-layout result may be less favorable than the synthesis result. This distinction should be explicitly discussed.

6.3 Area and energy trade-offs

Carry-save compression introduces additional arithmetic units and may increase local wiring. The results should therefore determine whether any latency reduction is achieved at an acceptable area and energy cost.

A fair comparison must use consistent cell libraries, timing constraints, switching activity assumptions, and physical-design settings. Energy results should state whether they include SRAM access, peripheral logic, registers, clock distribution, and the final output stage.

6.4 Workload evaluation

A BNN workload can test the architecture's intended use case. Suitable evaluation tasks include binary image classification and small convolutional networks.

The software reference implementation should produce expected dot products and inference outputs. Hardware results should then be compared against the reference to establish numerical correctness and determine whether the architecture changes inference accuracy.

Inference accuracy should be reported only when a defined model, dataset, training procedure, and evaluation protocol have been used.

7. Limitations

The proposed architecture has several limitations that require investigation.

First, carry-save reduction may not outperform an optimized tree of carry-propagate adders for every operand count or word width. Second, local reduction logic can increase area and wiring. Third, the final carry-propagate adder remains on the output path. Fourth, timing advantages measured before routing may disappear after parasitic extraction.

Finally, an architecture optimized for binary neural networks may not be optimal for multi-bit MAC operations. Extending the design to multi-bit precision requires a separate assessment of partial-product generation, accumulation width, and numerical accuracy.

8. Conclusion

This paper proposes hierarchical carry-save accumulation as a candidate method for reducing the accumulation-related latency of digital SRAM-CIM architectures. The method preserves SRAM-based XNOR computation while restructuring the accumulation network to postpone full carry propagation until the final addition.

The principal evaluation compares the proposed architecture with a baseline that uses the same memory-cell design and equivalent computational precision. Post-layout latency, energy, area, routing, and arithmetic correctness are necessary to establish whether the proposed method provides a practical advantage.

The architecture should be considered successful only if experiments demonstrate a reproducible improvement under comparable conditions. The magnitude of any improvement, its dependence on array size, and its suitability for binary neural network inference remain to be established experimentally.

References

[1] N. S. Dhakad and S. K. Vishvakarma, “SRAM Based Digital Custom Compute Engine for Improved Area Efficiency of AI Hardware,” arXiv:2605.16161, May 2026. doi: 10.48550/arXiv.2605.16161.

[2] “Low-Power and Area-Efficient CIM: An SRAM-based fully-digital computing-in-memory hardware acceleration processor with approximate adder tree for multi-precision sparse neural networks,” *Microelectronics Journal*, vol. 166, article 106903, Dec. 2025. doi: 10.1016/j.mejo.2025.106903.

[3] A. E. El Arrassi et al., “DREAM-CIM: A Digital SRAM-Based CIM Accelerator for Energy- and Area-Efficient Edge AI,” *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 2, no. 3, pp. 211–221, 2025.

[4] “An 8b-Precision 6T SRAM Computing-in-Memory Macro Using Time-Domain Incremental Accumulation for AI Edge Chips,” *IEEE Journal of Solid-State Circuits*, vol. 59, no. 7, pp. 2297–2309, July 2024. doi: 10.1109/JSSC.2023.3343669.

Reference details should be checked against the publisher or repository records before submission. Additional foundational references on carry-save arithmetic, digital SRAM-CIM, and adder-tree design should be added in the target journal's required citation style.

3. My assessment before submission

The draft gives you a workable starting point, but it is not yet a submission-ready SCI article. The key unresolved issue is novelty. Prior work already covers SRAM-CIM accumulation optimization, approximate adder trees, and custom accumulation structures.


ScienceDirect
+1

To make this a credible research paper, the next step should be to define the exact circuit topology and benchmark it against the reference design. I recommend:

1. Draw the proposed circuit architecture and specify how many partial-product operands each carry-save stage receives.
2. Prove arithmetic equivalence and determine the output bit width.
3. Implement the baseline and proposed designs with the same technology library and constraints.
4. Measure post-layout results and report actual latency, energy, area, and routing.
5. Expand the prior-art review before claiming the method is novel.

The title can remain One Method to Reduce the Latency in SRAM-Based Compute-in-Memory, although a more specific title may be stronger once the exact circuit contribution is established.